

PROJECT PLAN

VoicePilot

An AI-Powered Voice Assistant for Desktop Task Automation

DA598A – Introduction to Generative AI

Richmond Boakye

Individual project

1. Project Title

VoicePilot – An AI-Powered Voice Assistant for Desktop Task Automation

2. Group Member(s)

Richmond Boakye – Individual project

3. Objective

The objective is to build a browser-based voice assistant that lets a user control a small, safe set of desktop tasks using natural spoken language. The application will capture speech, convert it to text, interpret the user's intention with an AI model, and execute an approved action through a Python backend. It is intended for students, developers and other computer users who want a simple hands-free way to perform repetitive tasks such as opening an application, searching the web, creating a note, or recalling previously saved information. The project will demonstrate speech processing, LLM-based intent understanding, prompt design, conversation memory, data preprocessing and frontend/backend integration.

4. Application Features

- **Voice input and transcription:** Record a short command in the browser and transcribe it using a speech-to-text model.
- **Intent understanding:** Use an LLM to convert natural-language commands into a small structured action format, for example OPEN_APP, WEB_SEARCH, CREATE_NOTE or RECALL_MEMORY.
- **Safe desktop actions:** Execute only allow-listed actions such as opening selected applications/folders, opening a web search and creating a text note.
- **Conversation memory:** Store recent interactions and selected user notes so follow-up requests can use previous context.
- **Response generation:** Return a short natural-language confirmation or explanation after an action is interpreted or completed.
- **Browser interface:** Provide a simple React/TypeScript interface showing microphone status, transcript, detected intent, action result and conversation history.
- **Validation and error handling:** Reject unknown or unsafe actions and show clear feedback when speech or intent recognition fails.

5. Source of Data

The main project dataset will be a small raw voice-command dataset created for the project. I will record multiple variations of commands for each supported action, for example “Open Visual Studio Code”, “Start VS Code”, and “I want to code”. Each audio sample will be paired with a target intent label and, where relevant, an entity such as the application name or search query.

The raw data will require preprocessing before it is used. The planned preprocessing includes removing empty or unusable recordings, speech-to-text transcription, text normalization, duplicate checking, intent labeling, and splitting the samples into development/test groups. The processed data will be used to evaluate how reliably the assistant maps different natural-language expressions to the correct intent. A pretrained speech model will be used rather than training speech recognition from scratch.

If additional examples are needed, I may extend the dataset with manually written paraphrases of the supported commands, while keeping the original recorded samples as the primary raw source.

6. Intended Tech Stack

- **Frontend:** React, TypeScript, Vite, HTML/CSS and browser microphone APIs.
- **Backend:** Python, FastAPI and Uvicorn.
- **Speech-to-text:** Whisper or faster-whisper.
- **LLM / language understanding:** A small pretrained model accessed locally through Hugging Face or Ollama, with prompt refinement and structured output for intent detection.
- **Memory / persistence:** SQLite for conversation history, saved notes and basic application state.
- **Desktop integration:** Python standard libraries such as os, subprocess, pathlib and webbrowser, restricted to predefined allow-listed actions.
- **Development and testing:** Visual Studio Code, Git/GitHub, Postman or FastAPI Swagger for API testing, and pytest where appropriate.

Planned Project Scope

To keep the project feasible, the first version will support a limited set of approximately five to seven desktop intents rather than unrestricted computer control. The focus will be on a complete working pipeline: speech input → transcription → AI intent interpretation → validated action → response. Additional actions or text-to-speech output will be treated as optional extensions after the core functions are stable.